Learning Matches for Deformation via REINFORCE

Deniz Sayin Mercadier
CVLab, EPFL

Giorgio Panizzutti
EPFL

Sofia Sannino
EPFL

Abstract

Template-based mesh deformation is widely used in geometric deep learning and medical image analysis, where an initial source mesh is iteratively deformed to match a target shape. Standard pipelines usually rely on Chamfer distance, that can behave poorly as an optimization objective producing fragile correspondences and many-to-one vertex collapse. This project investigates whether point correspondences can instead be learned as a policy for iterative regularized deformation. We formulate mesh matching as a reinforcement learning problem, trying to learn a neural policy that can predict matches based on input shapes, in which the state is the current source-target configuration, the action is a correspondence proposal, the transition is represented by deformation steps, and the reward is the final fit quality. The REINFORCE algorithm allows us to optimize this discrete matching policy with policy gradients. As directly starting with 3D is challenging both technically and computationally, we begin with a simplified 2D environment using procedurally generated shapes, then compare the learned policy against nearest-neighbor Chamfer matches. We illustrate improvements and discuss various trade-offs, design choices and possible extensions supported by experiments. Our code can be accessed at: `https://github.com/denizmsayin/rl-mesh-deformation`

1 Introduction

Template-based deformation appears in geometric deep learning and medical image analysis when a fixed template mesh is adapted to match a target object. Fast mesh representation is required to easily model surface morphologies. For instance, researchers in biology and biomedical applications rely on biological imaging tools to study morphology of structures such as cells, organs, or tissues, and prefer to visualize them as 3D surfaces at the required level of detail everywhere. Moreover, even though volumes and distances can be estimated using voxels, analyzing surfaces is best accomplished using meshes. They have proven to be effective in segmenting, matching and tracking anatomic structures by exploiting (bottom-up) constraints derived from the image data together with (top-down) a priori knowledge about the location, size and shape of these structures [McInerney and Terzopoulos, 1996]. A template mesh is iteratively deformed to fit a target shape (e.g. a patient-specific organ) using a combination of Euclidean distance losses, such as Chamfer distance, and structural regularizers [Wickramasinghe et al., 2020]. A well-known failure mode of this optimization is that it can cause many-to-one vertex collapse, which the regularizer cannot undo, in particular when complex shapes are involved [Sayin Mercadier et al., 2025].

To overcome this obstacle, we investigated whether it is possible to learn a point correspondence policy, through REINFORCE [Sutton et al., 1999] algorithm, that improves iterative regularized deformation when standard nearest-neighbor matching can be unreliable under difficult initial conditions. In particular, we trained the model over 1,000,000 closed 2D shapes (circles, triangles, hexagon and n-point stars), aiming to learn an effective deformation policy from a source shape to a target shape, and then performed evaluation. Our reinforcement learning framework is structured in the following way:

Preprint.

1. The state is represented by the current source shape at deformation step h, $S_s^{(h)}$, and target shape S_t.
2. The action is represented by a correspondence proposal $C^{(h)}$ between $V_s^{(h)}$ and V_t.
3. The reward is represented by a final fit quality, measured with Chamfer distance.

Our framework is able to outperform one nearest-neighbor matching with Chamfer distance, effectively avoiding collapse modes, in the training and evaluation, although it can fail to capture finer details in simpler cases.

2 Related work

2.1 Deformable models in biomedical computer vision

Deformable surface models became popular in the 1990s to model biological structures in volumetric data [McInerney and Terzopoulos, 1996]. Later relevant work regards accurate reconstruction of organs and tissue surfaces from 3D volumes by mesh deformation [Wickramasinghe et al., 2020] [Zheng et al., 2023] [Teng et al., 2025]. In particular, Template-based mesh deformation is a technique in medical image analysis and geometric deep learning, in which a template mesh is iteratively deformed to fit a target shape (e.g. a patient-specific organ) through nearest neighbor using a combination of Euclidean distance losses and structural regularizers [Wickramasinghe et al., 2020]. A well-known failure mode of this optimization is that it can cause many-to-one vertex collapse, which the regularizer cannot undo [Song and Hyde, 2026]. We implemented a 2D reinforcement learning framework that models a deformation between meshes of 2D closed shapes (circles, polygons, n-point stars and more complex random shapes), learning matches replacing the nearest neighbor pipeline with a learned deformation policy, effectively reducing such degenerative cases.

2.2 Reinforcement learning in local features extraction

Local features have been a key computer vision technology since the introduction of SIFT [Lowe, 2004], enabling applications such as Structure-from-Motion (SfM) [Agarwal et al., 2009]. Given two images A and B with feature sets F_A and F_B, matching them has cost $O(|F_A| \cdot |F_B|)$. As each image pixel may become a feature, the problem quickly becomes intractable. A method to address this issue consists in implementing a reinforcement learning framework in which a probability map between extracted features is learned and matches between local features are sampled using REINFORCE algorithm [Tyszkiewicz et al., 2020], instead of relying on nearest-neighbor matching. We took inspiration from this approach to design our implementation. However, it is worth noting that the supervision signal is much stronger in this application due to the existence of ground truth matches as well as epipolar geometric consistency.

3 Method

3.1 Data generation

We represent each 2D shape as

$$S = (V, L), \tag{1}$$

where $V \in \mathbb{R}^{M\times 2}$ contains the M vertex coordinates and $L \in \mathbb{N}^{M\times 2}$ stores the M edges. For a deformation task, let

$$S_s^{(h)} = (V_s^{(h)}, L_s), \qquad S_t = (V_t, L_t) \tag{2}$$

be the source shape at deformation step h and the fixed target shape. Only the source vertices $V_s^{(h)}$ are updated, while the source connectivity L_s remains fixed.

The data are procedurally generated as closed 2D shapes, including polygons, hearts, n-point stars and more random shapes. Shape instances are varied through random rotations, translations, and anisotropic scaling. The simplified 2D setting allows us to test the matching idea before moving to full 3D meshes, where both computational limits and necessary algorithmic improvements are substantially harder.

3.2 Deformation Process

Given a correspondence proposal $C^{(h)}$, deformation is performed by stochastic gradient descent on an Euclidean matching loss plus a shape regularizer:

$$\mathcal{L}_{\text{deform}} = \mathcal{L}_{\text{match}}\left(V_s^{(h)}, C^{(h)}, V_t\right) + \lambda \mathcal{R}\left(V_s^{(h)}, L_s\right). \tag{3}$$

The matching loss moves each source point toward its assigned target point. The regularizer discourages degenerate deformations and helps preserve local shape structure. In the general iterative formulation, $C^{(h)}$ can be recomputed every K optimization steps and kept fixed in between. Chamfer-based deformation corresponds to choosing $C^{(h)}$ by nearest-neighbor search. Our method replaces this rule with a learned stochastic policy. We illustrate an example deformation process in Fig. 1.

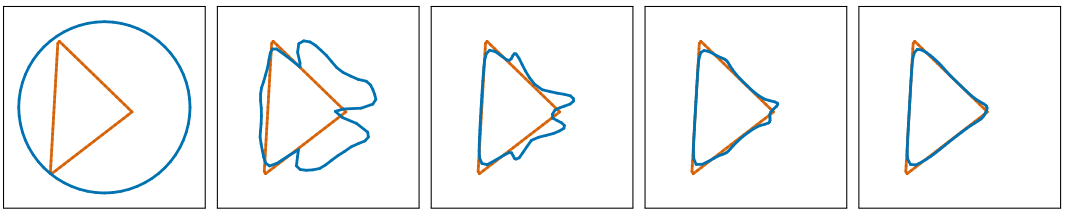


Figure 1: SGD deformation of a source circle (blue) toward a target triangle (orange), using 64 points, 300 iterations. Left to right: iterations 0, 75, 150, 225, and 300.

3.3 RL Formulation

We model the matching process as an episodic RL problem:

- **State:** the current source-target configuration $(S_s^{(h)}, S_t)$.
- **Action:** a correspondence proposal $C^{(h)}$ between $V_s^{(h)}$ and V_t.
- **Transition:** K SGD updates of Eq. 3 with the sampled correspondence fixed.
- **Reward:** final fit quality after deformation, measured as negative Chamfer distance.
- **Episode:** one complete deformation trajectory $S_s^{(0)} \to \cdots \to S_s^{(H)}$.

3.4 Learned correspondence policy

The standard Chamfer-based deformation chooses correspondences using nearest neighbors between the current source vertices and the target vertices. This gives a simple matching rule, but the matching itself is fixed and is not learned. Our goal is to replace such nearest-neighbor rule with a neural policy that predicts correspondences from the source and target shapes.

We use a shared feature extractor f_θ on both shapes. Given the source and target,

$$F_s = f_\theta(V_s, L_s) \in \mathbb{R}^{M_s \times D}, \qquad F_t = f_\theta(V_t, L_t) \in \mathbb{R}^{M_t \times D},$$

where each row is a feature vector for one vertex. We then compute a score matrix

$$P' = F_s F_t^\top \in \mathbb{R}^{M_s \times M_t}.$$

The entry P'_{ij} measures how compatible source vertex i is with target vertex j. We apply a row-wise softmax to obtain

$$P_{ij} = \frac{\exp(P'_{ij})}{\sum_{k=1}^{M_t} \exp(P'_{ik})}.$$

Each row of P is a probability distribution over target vertices for one source vertex.

During training, we sample one target index for each source vertex:

$$c_i \sim \text{Categorical}(P_{i,:}).$$

This gives a correspondence assignment

$$c = (c_1, \ldots, c_{M_s}).$$

The assignment is one-to-many, so different source vertices may choose the same target vertex. During evaluation, we use the most likely target vertex in each row instead of sampling.

3.5 REINFORCE training

The correspondence assignment is discrete, so we cannot train the matching choice directly with ordinary backpropagation. We therefore use the REINFORCE policy-gradient estimator. For each source-target pair, the policy samples a correspondence assignment c, the deformation process is run with this assignment fixed, and the final deformed source is compared to the target.

The reward is the negative Chamfer distance after deformation (H is the final deformation step):

$$R(c) = -d_{\mathrm{CD}}\big(S_s^{(H)}, S_t\big).$$

A better final fit gives a smaller Chamfer distance and therefore a larger reward.

The log-probability of the sampled correspondence is

$$\log \pi_\theta(c \mid V_s, V_t) = \sum_{i=1}^{M_s} \log P_{i,c_i}.$$

Therefore, the REINFORCE gradient can be written as

$$\nabla_\theta J(\theta) = \mathbb{E}_{c\sim\pi_\theta}\left[(R(c)-b)\nabla_\theta \log \pi_\theta(c \mid V_s, V_t)\right] = \mathbb{E}_{c\sim\pi_\theta}\left[(R(c)-b)\sum_{i=1}^{M_s}\nabla_\theta \log P_{i,c_i}\right].$$

where b is a baseline used to reduce variance. In the implementation, this expectation is estimated with one sampled correspondence per shape pair in a batch.

We also add a per-row entropy bonus to encourage exploration:

$$\beta\mathbb{E}\big[\mathcal{H}[\pi_\theta]\big].$$

For each source vertex, the policy defines a categorical distribution over target vertices. We compute the entropy of these row-wise distributions as

$$\mathcal{H}[\pi_\theta] = -\frac{1}{M_s}\sum_{i=1}^{M_s}\sum_{j=1}^{M_t} P_{ij}\log P_{ij}.$$

Thus, the policy loss used for minimization is

$$\mathcal{L}_{\mathrm{RL}} = -\big(R(c)-b\big)\sum_{i=1}^{M_s}\log P_{i,c_i} - \beta\mathcal{H}[\pi_\theta].$$

Here, β controls the strength of the entropy bonus. This encourages the correspondence policy to keep exploring different matches instead of becoming deterministic too early. In the general formulation, the matching may be recomputed every K deformation steps. In the current implementation the matching is sampled once at the beginning and kept fixed for all H deformation steps.

4 Setup

4.1 Deformation Process

The deformation process is driven by the following objective, given source shape $S_s = (V_s, L_s)$, target shape $S_t = (V_t, L_t)$ and matching C between points drawn from shapes $P_s \sim S_s$ and $P_t \sim S_t$, which is a set of pairs $i,\ j \quad \forall p_i \in P_s,\ \exists q_j \in P_t$:

$$\mathcal{L}_{\text{deformation}}(S_s, S_t, C) = \lambda_{\text{data}}\mathcal{L}_{\text{data}}(S_s, S_t, C) + \lambda_{\text{e}}\mathcal{L}_{\text{e}}(S_s) + \lambda_{\text{nc}}\mathcal{L}_{\text{nc}}(S_s) + \lambda_{\text{Lap}}\mathcal{L}_{\text{Lap}}(S_s) \quad (4)$$

The data term is the main driver of the deformation, optimizing the L_2 loss between the source and target point pairs based on the matching:

$$\mathcal{L}_{\text{data}}(S_s, S_t, C) = \frac{1}{M_s}\sum_{(i,\ j)\in C}\|p_i - q_j\|^2 \quad (5)$$

where $p_i \in S_s$ are the points lying on the source shape, and $q_j \in S_t$ are the point lying on the target shape. The other terms are various regularizers that operate only on S_s and attempt to keep its

structure from collapsing under the drive of the data term. $\mathcal{L}_{\text{e}}$ is the edge loss, that attempts to keep edges l_i of the source shape short:

$$\mathcal{L}_{\text{e}}(S_s) = \sum_{l_i \in L_s} \|l_i\|^2 \tag{6}$$

Similarly, the normal consistency loss $\mathcal{L}_{\text{nc}}$ tries to keep the normals of adjacent edges facing the same way for smoothness:

$$\mathcal{L}_{\text{nc}}(S_s) = \sum_{l_i \in L_s} \sum_{l_j \in L_s} \text{Adj}(l_i, l_j)(1 - \cos(n_{l_i}, n_{l_j})) \tag{7}$$

$$\text{Adj}(l_i, l_j) = \begin{cases} 1, \text{ if } l_i,\ l_j \text{ are adjacent edges} \\ 0, \text{ otherwise} \end{cases} \tag{8}$$

Finally, the Laplacian loss $\mathcal{L}_{\text{Lap}}$ tries to keep each vertex close to the average of its neighboring vertices:

$$\mathcal{L}_{\text{Lap}}(S_s) = \frac{1}{M_s} \sum_{i=1}^{M_s} \left\| v_i - \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} v_j \right\|^2 \tag{9}$$

where $\mathcal{N}(i)$ is the set of indices of vertices adjacent to v_i according to the source connectivity L_s. For our closed 2D polygons, $\mathcal{N}(i)$ contains the two neighbors of vertex i.

Varying the λ weights creates a trade-off between different terms of the loss. We use $\lambda_{\text{data}} = 1.0$, $\lambda_{\text{e}} = 1.0$, $\lambda_{\text{nc}} = 0.01$, $\lambda_{\text{Lap}} = 0.1$ unless stated otherwise in our experiments. While fully optimizing these smoothing losses to zero would lead to a flat shape with zero-length edges that has no purpose, their presence is meant to balance against the data loss term and preserve shape integrity by preventing degeneracies in the final result such as self-intersections.

We also set the learning rate to $\gamma = 0.0625$, and the number of iterations for the process to 400.

4.2 Network Architecture

While carrying over the architecture to more complex 2D structures or 3D requires graph neural networks [Bronstein et al., 2021], we use a 1-dimensional convolutional neural network for prototyping as our dataset is composed of polygons. These can be seen as a particular and more computationally efficient case of the more general graph neural network. We use a simple, three layer network with an input dimension of 2 (the coordinates), and output dimensions of 32, 64 and 128 respectively. The kernel size is 5 and stride 1. We circularly pad the input to obtain the same number of outputs as inputs (one per point), and in the same order. We use ReLU as the activation function and layer normalization [Ba et al., 2016] before each activation. We optimize the network using the Adam optimizer with learning rate $\gamma = 10^{-4}$.

4.3 Dataset

Our dataset is composed of 2D shapes. We stick to a simple circle as the source shape in various configurations, since it's expected to start the deformation from a simpler template. Our target shapes are randomly distributed between circles, hexagons, triangles and 5-point stars as a small but diverse target. All shapes are initially generated with a scale approximately fitting into the unit cube. We then augment them by randomly rotating up to 360°, scaling and translating. The scaling is uniformly isotropic in the $[0.5, 2.0]$ range for the source circle, and anisotropic in the $[0.2, 2.0]$ range for the target shapes. Translation is uniform in both axes in the $[-1, 1]$ range. We show some sample pairs in Fig. 2. All shape pairs are uniformly resampled to $M = 64$ points in our experiments unless stated otherwise. During model training, shape pairs are continuously drawn from the random augmented distribution with fixed seeds for reproducibility. For evaluation, we use a separate fixed set of 2,000 generated shape pairs to ensure consistency between different tests.

5 Experiments

Our initial experiment uses the settings described so far, and is trained on 2,000,000 random deformation trajectories. On the evaluation set, we obtain the metrics shown in Tab. 1, where normal

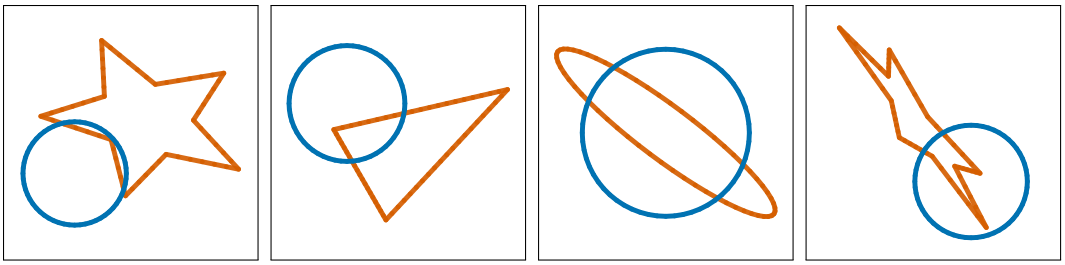


Figure 2: Random source (blue) and target (orange) shape pairs from the training dataset.

consistency is an extra metric in the $[0, 1]$ range that improves when surface normals between two shapes match. Our results show that our model succeeds in avoiding catastrophic collapses that can happen when using Chamfer distance only, but can miss high-frequency details. This is illustrated in Fig. 3. We expect this due to our formulation not rewarding the reproduction of these details very well.The results also seem to support our initial premise: **Chamfer distance works better as a metric than an optimization objective.** A direct optimization of Chamfer distance produces worse results in terms of Chamfer distance than a learned matching, due to matches themselves not being learnable without our RL formulation.

Metric	**Chamfer Def.**	**Learned Def.**
Chamfer distance ↓	7.351×10^{-3}	$\mathbf{6.732 \times 10^{-3}}$
Normal consistency ↑	0.923	**0.966**

Table 1: Metrics averaged over evaluation samples between the two types of deformation.

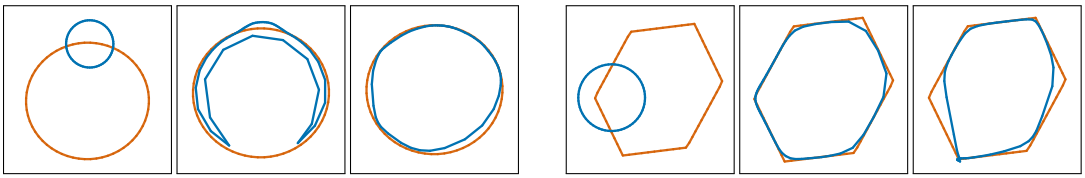


Figure 3: Deformation comparison on two evaluation pairs.

5.1 Effect of the number of samples M

One significant limitation of the method is that a higher M value (number of points in resampled source and target shapes) requires more training due to the larger match sampling space and is less stable. We show training curves with the average reward over time for $M = 16$, 64 and 256 in Fig. 4.

5.2 Baselines

We tested out three baselines b for our model:

- The first was the "untrained" baseline, where we used an uninitialized version of the trained model, which we thought might prevent the model from "slipping" below its initial configuration.
- Another was an exponential moving average (EMA) with $\alpha = 0.9$ where we kept a running average of previous rewards as baseline.
- Finally, we tried directly using the Chamfer distance deformation process as a baseline, thinking that the model could quickly converge by being forced to improve over it, despite this being a computationally expensive approach.

For $M = 16$ and 1M trajectories, we show the average collected reward in Fig. 5a. The untrained baseline underperforms, which is understandable due to high variance, other baselines show almost

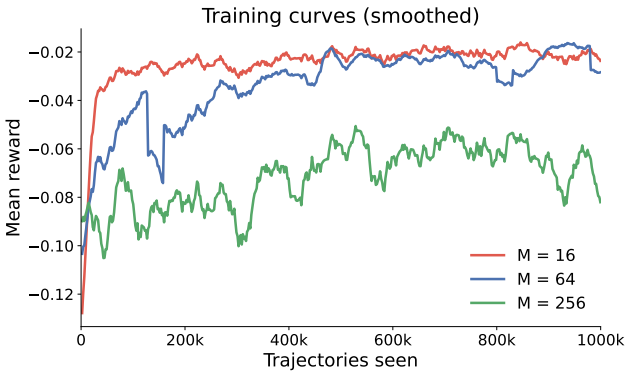
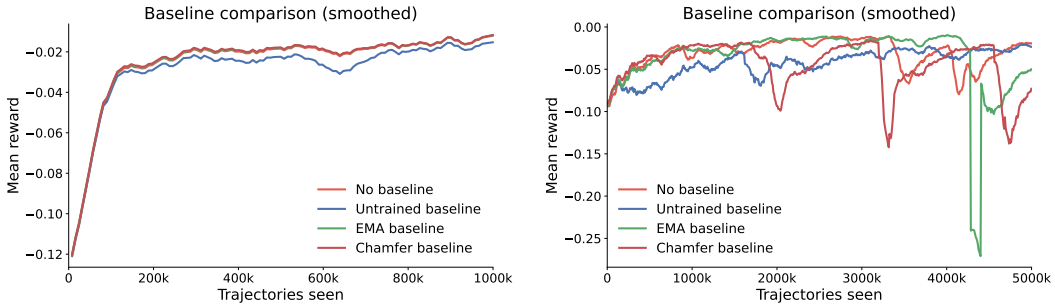


Figure 4: Reward curves over time for different M values. Convergence worsens with higher M.

no change in behavior. To improve our understanding, we tried $M = 128$ and 5M trajectories for higher variance, shown in Fig. 5b. There, we see the Chamfer baseline slightly outperforming the no baseline version, but this is not by a significant margin. We noted another limitation where training becomes unstable at large number of trajectories, with both EMA and Chamfer baselines slip back to low rewards. We believe this might be a limitation of our raw REINFORCE approach, and it might be improved using methods like Proximal Policy Optimization [Schulman et al., 2017], where the policy is anchored more stably by clipped updates and its variance reduced with a learned neural network baseline.



(a) (b)

Figure 5: Short, low variance ($M = 16$) run (a) of baselines compared with a longer, high variance run ($M = 256$) (b).

5.3 Supplementary Videos

As deformations are best followed visually, we show some example Chamfer deformations in `videos/basic_chamfer.mp4`, and learned ones in `videos/basic_learned.mp4`.

Another interesting result we found is the fact that the learning approach can find relatively stable matches even in an extremely overtuned, oscillatory scenario with $\lambda_{\text{data}} = 100.0$ rather than 1.0 where Chamfer tends to collapse the shape heavily. These can be seen in `videos/extreme_chamfer.mp4` and `videos/extreme_learned.mp4`.

5.4 Further Experiments

We performed a few more sets of experiments as stepping stones towards 3D, but our results in these ended up being limited and the development not fully complete. We still discuss them for future reference.

Re-matching every K iterations: Rather than using our simplified bandit formulation where we match only once at the beginning, we tried to learn a model which re-matches every K iterations,

and set $K = 80$, meaning we re-match five times. The most important consideration here is reward assignment, as we now have multiple steps relying on the final Chamfer reward. We tried two formulations: a **joint** assignment where we take each match as independent and simply assign the same final reward to each of them, and a **progressive** assignment where we assign the "improvement" as reward, i.e. the difference in reward at the point where the match was made and at the end, to assign more weight to more important matching steps. In the end, we ended up with a final average reward of 0.23 for the joint assignment and 0.05 for the progressive one, whereas the base approach where we match once leads to around 0.02. Outperforming the match-once approach requires further design consideration.

Changing to a graph neural network: As a step towards generalizing the model to work with more complicated topologies, we wanted to change our network from a 1D CNN to a graph network. As an initial step, we used a basic architecture similar to EdgeConv [Wang et al., 2019], and tried aggregating neighbor activations via both mean and max. This network severely underperformed the 1D CNN, reaching a final average reward of around 0.10 compared 0.02 and had a tendency to increase entropy heavily compared to the CNN. We believe this to be due of the permutation-invariant and equally-weighted architecture which reduces discriminative power compared to the well-ordered basic polygon CNN. More difficult topologies will have to come after the careful design of a graph network that can catch up to the 1D CNN performance.

6 Limitations, Future Work and Conclusion

While our prototyping foray into using reinforcement learning was relatively successful and showed that matches for a deformation process can be learned well even in a single-match setting, carrying it over to 3D and then a real application such as medical imaging requires solving further challenges:

- Naturally, many more control points M are required for accurate 3D deformation compared to 2D due to the curse of dimensionality. Our experiments showed that a larger M makes convergence more difficult, so mitigation strategies are required. While even basic algorithmic improvements like the use of PPO [Schulman et al., 2017] can help, more drastic measures for reducing the match space such as using a masking strategy based on orientation, or using a hierarchical approach where we move from coarser matches to denser matches with a conditional neural network will probably be a necessity.
- Reward design has a significant effect on results. While basic Chamfer was promising, even small additions like normal consistency help results. The fact that the reward does not need to be differentiable is another bonus, and alternative reward formulations will need to be considered.
- Real pipelines use very aggressive deformations that are completed in a few steps with neural networks, with Chamfer distance only being used during training as a supervision signal. Trying to improve the supervision signal would require training an RL matcher in advance. Resource and dataset considerations apply, especially when considering sparse training data cases like medical imaging. The design of a clever end-to-end pipeline that could improve upon Chamfer distance as well as more computationally expensive alternatives like Sinkhorn distance [Cuturi, 2013] while remaining tractable and efficient is an open problem.
- Compared to supervised learning, reinforcement learning appears quite brittle and there are many possibilities for things to go wrong. Even having this simple working prototype required careful design on our part and wading through significant bugs.

Despite these limitations, we had a lot of fun with this project. Overcoming fundamental limitations like gradient flow through fixed nearest-neighbor matches by using sampling and learning by only improving this sampling distribution with a non-differentiable reward was quite eye opening! Tackling one or more of the above limitations will almost certainly be framed soon as a semester project in CVLab for interested parties.

Many thanks to the efforts of the EE-568 staff for this interesting journey!

References

Sameer Agarwal, Noah Snavely, Ian Simon, Steven M. Seitz, and Richard Szeliski. Building rome in a day. In *2009 IEEE 12th International Conference on Computer Vision*, pages 72–79. IEEE, 2009. doi: 10.1109/ICCV.2009.5459148.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Michael M Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.

Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In C.J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL `https://proceedings.neurips.cc/paper_files/paper/2013/file/af21d0c97db2e27e13572cbf59eb343d-Paper.pdf`.

David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, November 2004. doi: 10.1023/B:VISI.0000029664.99615.94.

Tim McInerney and Demetri Terzopoulos. Deformable models in medical image analysis: A survey. *Medical Image Analysis*, 1(2):91–108, 1996. doi: 10.1016/S1361-8415(96)80007-7.

Deniz Sayin Mercadier, Hieu Le, Yihong Chen, Jiancheng Yang, Udaranga Wickramasinghe, and Pascal Fua. Printmesh: Precise intersection surfaces for 3d organ mesh reconstruction. *arXiv preprint arXiv:2511.16186*, 2025.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Chang-Yong Song and David Hyde. On the structural failure of chamfer distance in 3d shape optimization. *arXiv preprint arXiv:2603.09925*, 2026.

Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, volume 12, 1999.

Lin Teng, Shen Zhao, Feng Shi, and Dinggang Shen. A curvature-guided diffeomorphic mesh deformation framework for lifespan brain cortical surface reconstruction. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2025*, volume LNCS 15960, pages 14–23. Springer Nature Switzerland, 2025. doi: 10.1007/978-3-032-04927-8_2. URL `https://papers.miccai.org/miccai-2025/paper/1603_paper.pdf`.

Michał Tyszkiewicz, Pascal Fua, and Eduard Trulls. Disk: Learning local features with policy gradient. In *Advances in Neural Information Processing Systems*, volume 33, pages 14254–14265, 2020.

Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics (tog)*, 38(5): 1–12, 2019.

Udaranga Wickramasinghe, Edoardo Remelli, Graham Knott, and Pascal Fua. Voxel2mesh: 3d mesh model generation from volumetric data. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 299–308. Springer, 2020.

Hao Zheng, Hongming Li, and Yong Fan. Coupled reconstruction of cortical surfaces by diffeomorphic mesh deformation. In *Advances in Neural Information Processing Systems*, volume 36, pages 80608–80621. Curran Associates, Inc., 2023. URL `https://papers.neurips.cc/paper_files/paper/2023/hash/ff0da832a110c6537e885cdfbac80a94-Abstract-Conference.html`.